

Proyecto Juegos Reunidos: Dominó



Título del proyecto: Juegos reunidos - Dominó

Grupo: 25

Componentes del grupo: Álvaro Santos, Nicolás García-Sampedro y Chao An Alarcón

Titulación: DAW

Curso: 1º

Periodo docente: 2024-25

1. Contexto Inicial

El objetivo de este proyecto es desarrollar una versión digital del juego clásico de Dominó. Para ello, se aplicarán conceptos de programación orientada a objetos (POO), siguiendo buenas prácticas y garantizando que el código sea modular, escalable y reutilizable.

A lo largo del desarrollo se reforzarán habilidades como la abstracción, encapsulación y uso de estructuras de datos adecuadas, además del trabajo en equipo y la planificación de proyectos.

2. Toma de Requisitos

2.1 Requisitos Funcionales

- Implementar la lógica del juego de Dominó.
- Permitir jugar contra otro/s jugador/es en el mismo dispositivo.
- Incluir un sistema de turnos y verificación de jugadas válidas.
- Implementar un menú con las opciones:
 1. Jugar Partida
 2. Ver Instrucciones
 3. Salir

2.2 Diagrama de Clases

Clase FichaDomino

- Atributos:
 - lado1: int → Primer número de la ficha.
 - lado2: int → Segundo número de la ficha.
 - temporal: int → Variable temporal para girar la ficha.
- Métodos:
 - getLado1(): int → Devuelve el valor del primer lado.
 - setLado1(lado1: int): void → Establece el valor del primer lado.
 - getLado2(): int → Devuelve el valor del segundo lado.
 - setLado2(lado2: int): void → Establece el valor del segundo lado.
 - esDoble(): boolean → Comprueba si es una ficha doble.
 - voltear(): void → Invierte los valores de la ficha.
 - toString(): String → Representación textual de la ficha.

Clase ConjuntoFichas

- Atributos:
 - fichas: ArrayList<FichaDomino> → Conjunto de todas las fichas.
- Métodos:
 - ConjuntoFichas() → Constructor que inicializa las 28 fichas.
 - barajar(): void → Mezcla aleatoriamente las fichas.
 - sacarFicha(): FichaDomino → Cuando un jugador no tiene una ficha para jugar, extrae una de las que no se repartieron.

- `estaVacio()`: boolean → Verifica si quedan fichas disponibles para poder robar.

Clase NumeroJugadores

- Atributos:
 - `numJugadores`: int → Cantidad de jugadores en la partida.
 - `jugadores`: ArrayList<Jugador> → Lista de todos los jugadores.
 - `sc`: Scanner → Para entrada de usuario.
- Métodos:
 - `NumeroJugadores(numJugadores: int)` → Constructor con número de jugadores.
 - `NumeroJugadores()` → Constructor por defecto.
 - `getNumJugadores(): int` → Devuelve el número de jugadores.
 - `setNumJugadores(numJugadores: int): void` → Establece el número de jugadores.
 - `establecerNumeroJugadores(): int` → Solicita y configura el número de jugadores.

Clase Jugador

- Atributos:
 - `nombre`: String → Nombre del jugador.
 - `indice`: int → Índice de ficha seleccionada.
 - `lado`: String → Lado seleccionado para colocar ficha.
 - `extremoIzq`: int → Extremo izquierdo del tablero.
 - `extremoDer`: int → Extremo derecho del tablero.
 - `fichaColocada`: boolean → Indica si la ficha fue colocada.
 - `indiceInicial`: int → Índice de la ficha inicial.
 - `mano`: ArrayList<FichaDomino> → Fichas que tiene el jugador.
 - `FichaDomino` ficha;
 - `sc`: Scanner → Para entrada de usuario.
- Métodos:
 - `Jugador(nombre: String)` → Constructor con nombre del jugador.
 - `getNombre(): String` → Devuelve el nombre del jugador.
 - `setNombre(nombre: String): void` → Establece el nombre del jugador.
 - `getIndiceInicial(): int` → Devuelve el índice de ficha inicial.

- setIndiceInicial(indiceInicial: int): void → Establece el índice inicial.
- puedeJugar(tablero: ArrayList<FichaDomino>): boolean → Verifica si puede jugar.
- jugarFicha(tablero: ArrayList<FichaDomino>, sc: Scanner): void → Coloca una ficha.

Clase Juego

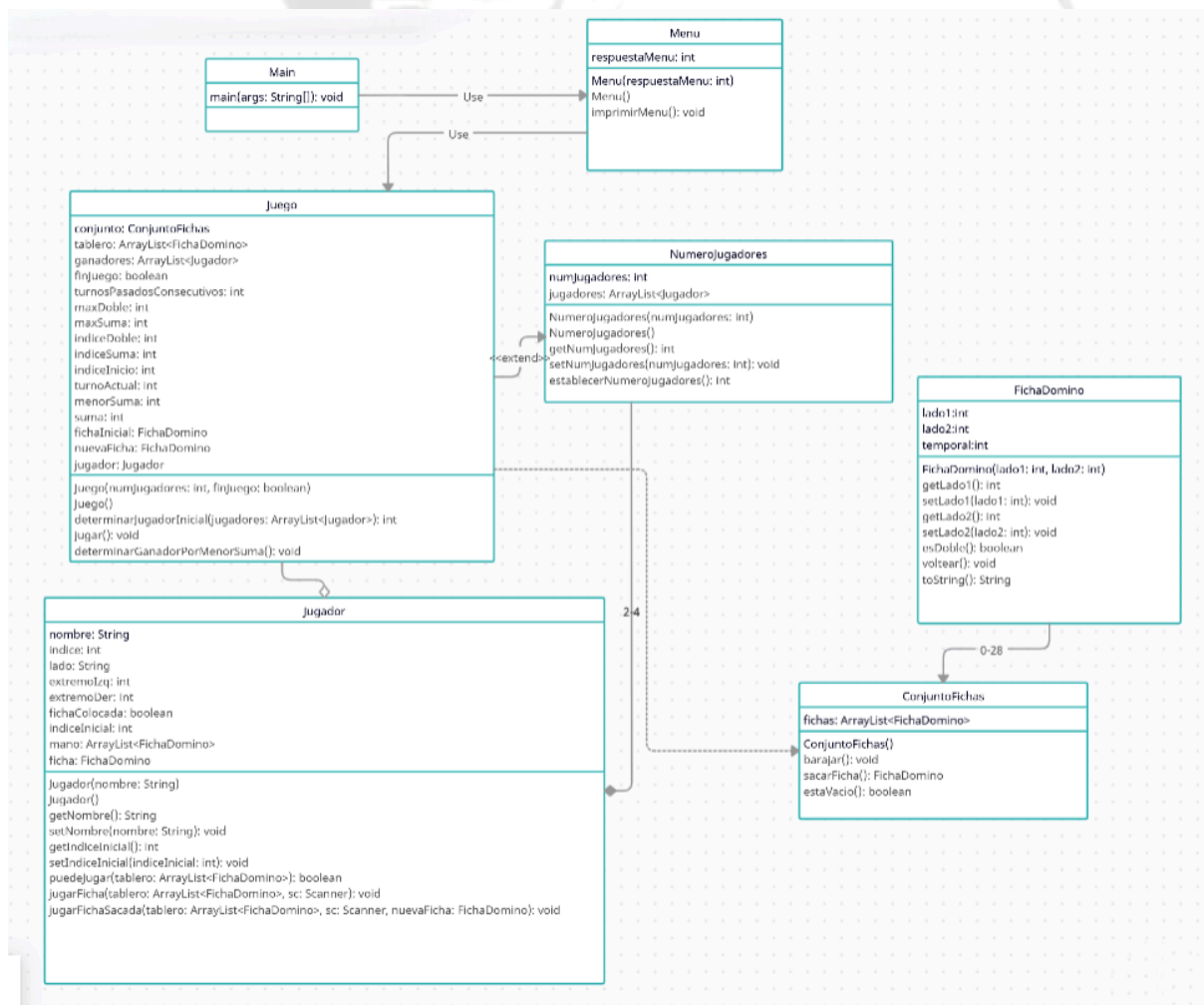
- Atributos:
 - sc: Scanner → Para entrada de usuario.
 - conjunto: ConjuntoFichas → Conjunto de fichas disponibles.
 - ArrayList<FichaDomino>Tablero → Fichas colocadas en el tablero.
 - ArrayList<Jugador> ganadores → Lista de ganadores por si hubiera un empate
 - finJuego: boolean → Indica si el juego ha terminado.
 - turnosPasadosConsecutivos: int → Conteo de turnos pasados.
 - maxDoble: int → Valor máximo de ficha doble.
 - maxSuma: int → Valor máximo de suma de ficha.
 - indiceDoble: int → Índice del jugador con máxima doble.
 - indiceSuma: int → Índice del jugador con máxima suma.
 - indiceInicio: int → Índice del jugador que inicia.
 - turnoActual: int → Índice del jugador actual.
 - menorSuma: int → Menor suma de fichas para desempate.
 - suma: int → Variable para calcular suma de fichas.
 - FichaDomino fichaInicial;
 - FichaDomino nuevaFicha;
 - Jugador jugador;
- Métodos:
 - Juego(numJugadores: int, finJuego: boolean) → Constructor con parámetros.
 - Juego() → Constructor por defecto.
 - determinarJugadorInicial(jugadores: ArrayList<Jugador>): int → Determina quién inicia.
 - jugar(): void → Gestiona el flujo completo del juego.
 - determinarGanadorPorMenorSuma(): void → Determina ganador por menor suma.

Clase Menu

- Atributos:
 - sc: Scanner → Para entrada de usuario.
 - respuestaMenu: int → Opción seleccionada por el usuario.
- Métodos:
 - Menu(respuestaMenu: int) → Constructor con respuesta del menú.
 - Menu() → Constructor por defecto.
 - toString(): String → Devuelve las instrucciones del juego.
 - imprimirMenu(): void → Muestra el menú y procesa la selección.

Clase Main

- Métodos:
 - main(args: String[]): void → Punto de entrada del programa



Composición: Jugadores es composición de NumeroJugadores ya que en este se eligen el número de personas que van a jugar y se añaden al array de la clase Jugadores

Agregación: Jugador es agregación de Juego porque en la clase Jugador tienen el array de mano en el que cada jugador tiene sus fichas y pueden elegir las fichas que quieran jugar.

Dependencia: Juego depende de ConjuntoFichas al tener que barajar las fichas y para que un jugador pueda sacar nuevas fichas.

Herencia: Juego hereda el número de jugadores de la clase de NumeroJugadores para gestionar los turnos de la partida.

Asociación: Main usa la clase menú para imprimir el menú. Menú usa juego para empezar a jugar.

3. Planificación de escenario de Juego

Fichas Dominó Planificación

Variables:

- Lado izquierdo
- Lado derecho
- 0-6 (Ambos lados)

Fichas:

```
[[0|0], [0|1], [0|2], [0|3], [0|4], [0|5],  
[0|6], [1|1], [1|2], [1|3], [1|4], [1|5],  
[1|6], [2|2], [2|3], [2|4], [2|5], [2|6],  
[3|3], [3|4], [3|5], [3|6], [4|4], [4|5],  
[4|6], [5|5], [5|6], [6|6]]
```

Planificación juego (7 fichas cada jugador)

Estas son vuestras fichas:

Jugador 1:

Jugador 2:

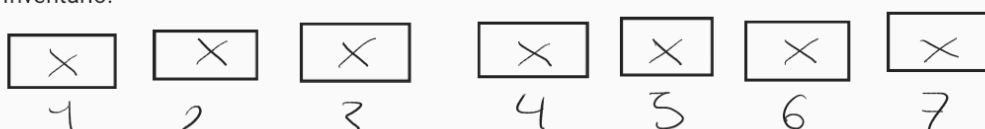
Empieza la partida:

Tablero :

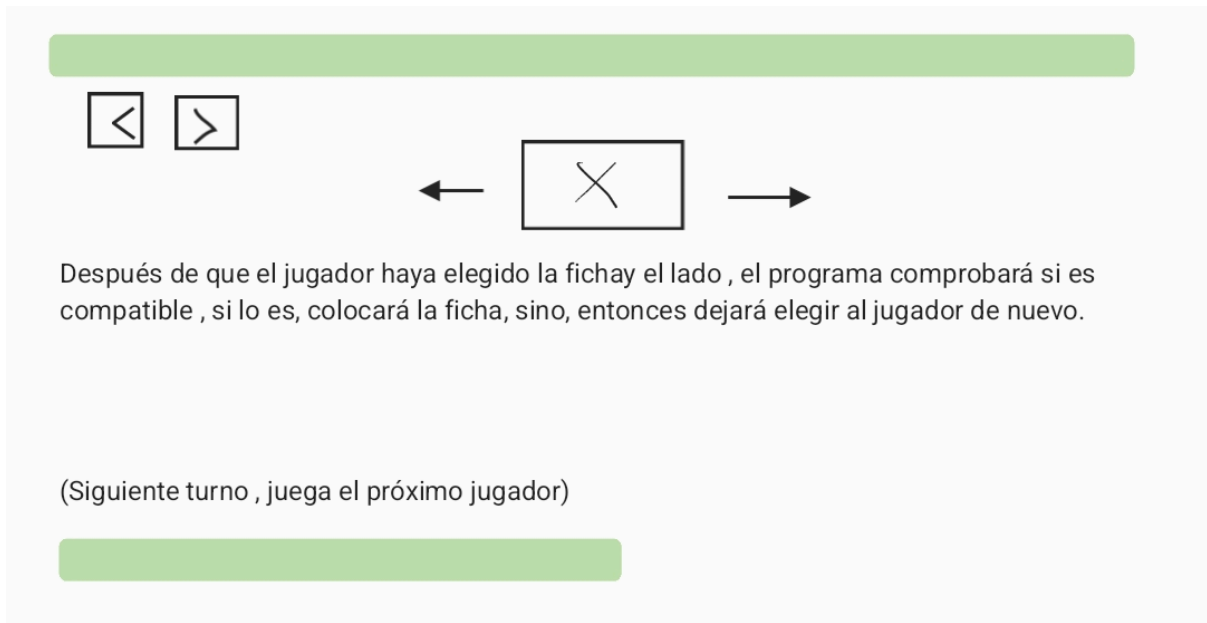


Turno jugador 1:

Inventario:



(Elige unatecla numérica en función de la ficha)



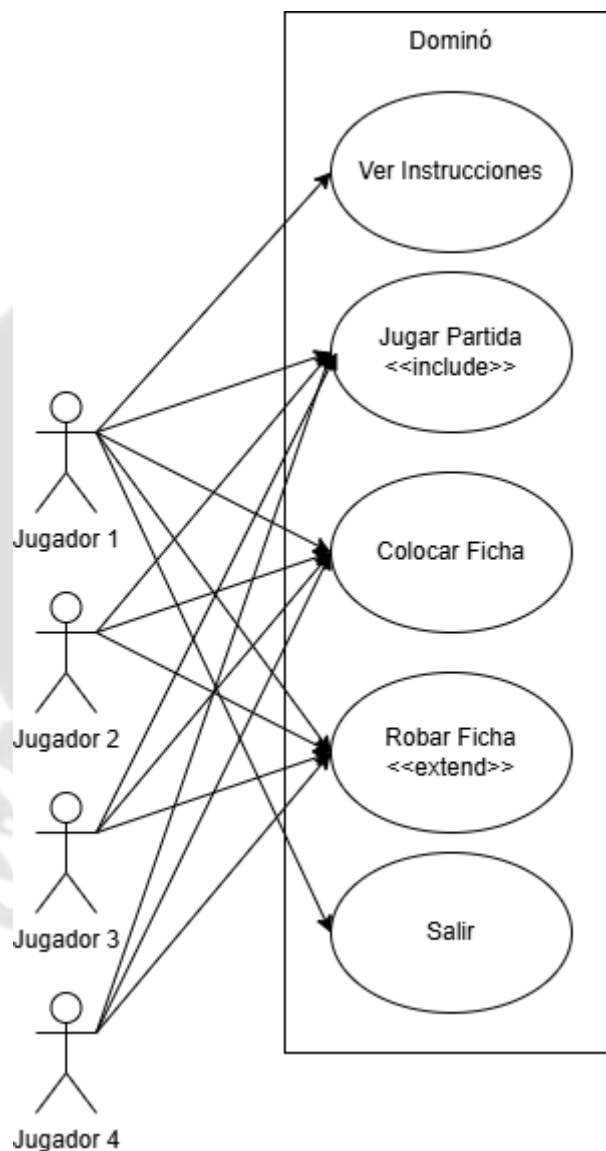
4. Diseño

4.1 Diseño Orientado a Objetos

Para el desarrollo del Dominó, hemos planteado una estructura basada en las siguientes clases:

- **Ficha Dominó:** Representa una pieza del dominó con dos valores.
- **Jugador:** Gestiona las fichas de un jugador y sus movimientos.
- **Juego:** Administra la partida, turnos y lógica general.
- **Interfaz:** Muestra el menú y gestiona la interacción con el usuario.
- **Número de Jugadores:** Gestiona el número de personas que van a jugar
- **Conjunto Fichas:** Crea y baraja las fichas.

4.2 Diagrama de Casos de Uso



5. Implementación

Aplicamos el paradigma de Programación Orientada a Objetos (POO). Utilizamos **ArrayList** para gestionar las fichas en el tablero, lo cual facilitó la modificación dinámica de su estructura. Además, seguimos un enfoque modular para mejorar la mantenibilidad del código.

5.1 Aspectos Clave Implementados

- **Encapsulación:** Hemos definido atributos privados y métodos de acceso.

```
package main;

import java.util.*;

public class Jugador {
    //Atributos
    Scanner sc = new Scanner(System.in);
    private String nombre;
    private int indice;
    private String lado;
    private int extremoIzq;
    private int extremoDer;
    private boolean fichaColocada = false;
    private int indiceInicial;
    ArrayList<FichaDomino> mano;
    FichaDomino ficha;
```

```
//Constructores
public Jugador(String nombre) {
    this.nombre = nombre;
    this.mano = new ArrayList<>();
    this.indiceInicial = 0;
}

public Jugador() {
}

//Getters y Setters
public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public int getIndiceInicial() {
    return indiceInicial;
}

public void setIndiceInicial(int indiceInicial) {
    this.indiceInicial = indiceInicial;
}
```

- **Herencia y Composición:** Hemos utilizado la composición para estructurar las clases correctamente.

```
//Otros métodos
public int establecerNumeroJugadores() {
    System.out.print("Ingrese el número de jugadores (2-4): ");
    numJugadores = sc.nextInt();
    while (numJugadores < 2 || numJugadores > 4) {
        System.out.println("\nEso no es una respuesta válida.");
        System.out.print("Ingrese el número de jugadores (2-4): ");
        numJugadores = sc.nextInt();
    }
    //Se añade el número elegido de jugadores a la lista de jugadores
    for (int i = 0; i < numJugadores; i++) {
        jugadores.add(new Jugador("Jugador " + (i + 1)));
    }
    return numJugadores;
}
```

```
• public ConjuntoFichas() {
    fichas = new ArrayList<>();
    for (int i = 0; i <= 6; i++) {
        for (int j = i; j <= 6; j++) {
            fichas.add(new FichaDomino(i, j));
        }
    }
}

• public void barajar() {
    Collections.shuffle(fichas);
}

• public FichaDomino sacarFicha() {
    return !fichas.isEmpty() ? fichas.remove(0) : null;
}

• public boolean estaVacio() {
    return fichas.isEmpty();
}
}
```

```
package main;

public class FichaDomino {
    //Atributos
    private int lado1;
    private int lado2;
    private int temporal;

    //Constructores
    public FichaDomino(int lado1, int lado2) {
        this.lado1 = lado1;
        this.lado2 = lado2;
    }
}
```

```
//Otros métodos
public boolean esDoble() {
    return lado1 == lado2;
}

public void voltear() {
    temporal = lado1;
    lado1 = lado2;
    lado2 = temporal;
}

public String toString() {
    return "[" + lado1 + "|" + lado2 + "]";
}
```

```
public class Juego extends NumeroJugadores {
    Scanner sc = new Scanner(System.in);
    ConjuntoFichas conjunto = new ConjuntoFichas();
    ArrayList<FichaDomino> tablero = new ArrayList<>();
    private boolean finJuego = false;
    private int turnosPasadosConsecutivos = 0;
    private int maxDoble = -1;
    private int maxSuma = -1;
    private int indiceDoble = -1;
    private int indiceSuma = -1;
    private int indiceInicio;
    private int turnoActual;
    private int menorSuma;
    private int suma;

    public Juego(int numJugadores, boolean finJuego) {
        super(numJugadores);
        this.finJuego = finJuego;
    }

    public Juego() {
    }
}
```

```
//Metodo para establecer el jugador inicial
private int determinarJugadorInicial(ArrayList<Jugador> jugadores) {
    for (int i = 0; i < jugadores.size(); i++) {
        int j = -1;
        suma = 0;
        for (FichaDomino ficha : jugadores.get(i).mano) {
            j++;
            // Se busca la ficha doble con el mayor valor y asigna al que la tiene como jugador inicial
            if (ficha.esDoble() && ficha.getLado1() > maxDoble) {
                maxDoble = ficha.getLado1();
                indiceDoble = i;
                jugadores.get(i).setIndiceInicial(j);
            }
            // Si no hay dobles, se busca la ficha con la mayor suma de lados y se asigna al que la tiene como jugador inicial
            suma = ficha.getLado1() + ficha.getLado2();
            if (suma > maxSuma) {
                maxSuma = suma;
                indiceSuma = i;
            }
        }
    }
    //SI HAY JUGADORES CON DOBLES, SE DEVUELVE EL ÍNDICE DEL QUE TENGA EL DOBLE MÁS ALTO, SI NO, SE DEVUELVE EL ÍNDICE DEL QUE TENGA LA FICHA CON MAYOR SUMA
    return (indiceDoble != -1) ? indiceDoble : indiceSuma;
}
```

- **Modularización:** Cada funcionalidad está separada en diferentes clases para evitar código repetitivo.
 - **Gestión de Turnos:** Hemos implementado una estructura de control para alternar entre jugadores y validar movimientos.
-

6. Lecciones Aprendidas

Durante el desarrollo del proyecto, nos hemos enfrentado diversos retos, como la gestión de los turnos de los jugadores y la validación de movimientos. Estos problemas los resolvimos con pruebas iterativas y optimización del código.

Además, la planificación y trabajo en equipo han sido fundamentales para dividir tareas y asegurar la cohesión del proyecto. Se han adquirido habilidades clave que podrán aplicarse en futuros desarrollos.

7. Mejoras a Futuro:

- **Interfaz gráfica:** Incorporar una interfaz gráfica más amigable para llegar a más público.
- **Base de Datos:** Utilizar una base de datos para almacenar las partidas, estadísticas de jugadores, y otros datos relevantes puede ayudar a escalar el juego para un mayor número de usuarios.
- **Despliegue en la Nube:** Considerar desplegar el juego en una plataforma de nube como AWS o Azure, lo que permite escalar los recursos según la demanda.